



Chap 03 운영체제_3.3 프로세스와 스레드

3.3 프로세스와 스레드

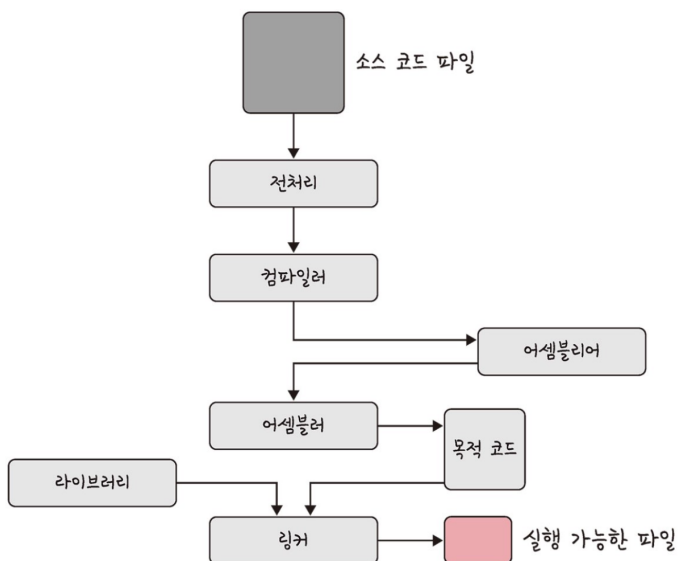
프로세스란?

컴퓨터에서 실행되고 있는 프로그램
CPU 스케줄링의 대상이 되는 작업

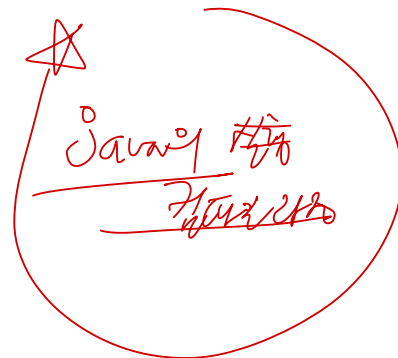
스레드란?

프로세스 내 작업의 흐름

3.3.1 프로세스와 컴파일 과정



▲ 그림 3-17 프로그램의 컴파일 과정



✅ 1단계) 전처리

소스 코드의 주석을 제거하고 #include 등 헤더 파일을 병합하여 매크로를 치환

✓ 2단계) 컴파일러

오류 처리, 코드 최적화 작업을 하며 어셈블리어로 변환

✓ 3단계) 어셈블러

어셈블리어의 목적 코드(object code)로 변환 (어셈블리어 소스 코드 → 기계어)

이때 확장자는 OS마다 다름 ex) 리눅스 : .o

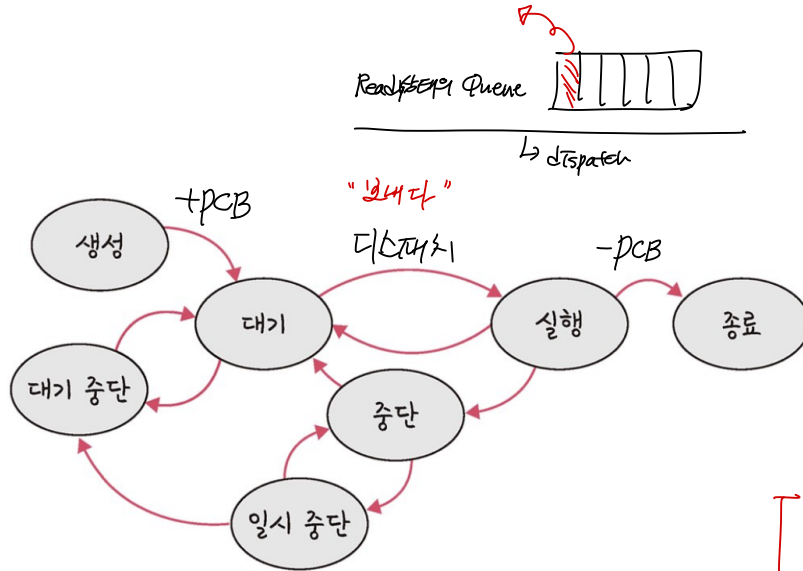
✓ 4단계) 링커

프로그램 내에 있는 라이브러리 함수 또는 다른 파일들과 목적 코드를 결합하여 실행 파일을 만들
실행 파일 확장자 : .exe or .out

정적 라이브러리 vs 동적 라이브러리

	정적 라이브러리	동적 라이브러리
개념	프로그램 빌드 시 라이브러리가 제공하는 모든 코드를 실행 파일에 넣는 방식으로 라이브러리 사용	프로그램 실행 시 필요할 때만 DDL이라는 함수 정보를 통해 참조하여 라이브러리 사용
특징	- 시스템 환경 등 외부 의존도 낮음 - 코드 중복 등 메모리 효율성 떨어짐	- 메모리 효율성 좋음 - 외부 의존도 높음

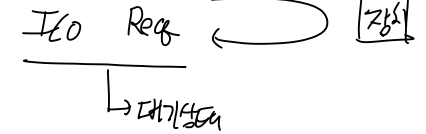
3.3.2 프로세스 상태



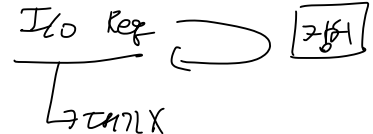
▲ 그림 3-18 프로세스의 상태

"인식자의 관점"

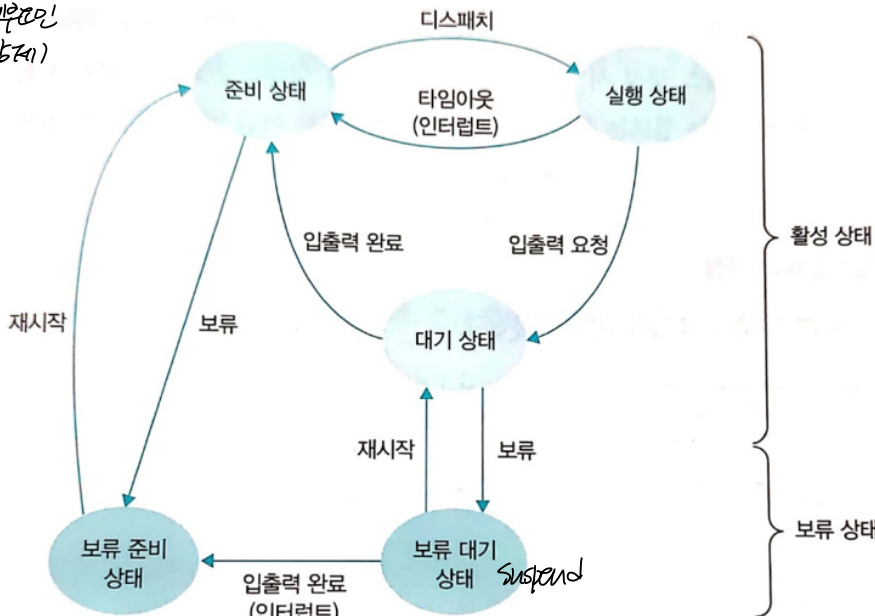
Blocking IO



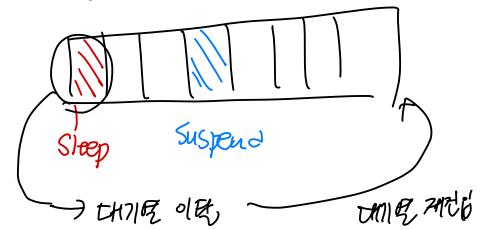
UnBlocking IO



Sleep : 자발적
suspend : 외부인 (외부)



Ready Queue



Sleep() $\Rightarrow$ Time t

앞에 대기 process 기다림

✓ 생성 상태 (Create)

= 프로세스가 생성된 상태

= fork(), exec() 함수를 통해 생성 $\rightarrow$ 이때 PCB 할당됨

프로세스가 하나 생성됨

*fork() : 부모 프로세스의 주소 공간을 그대로 복사해 새로운 자식 프로세스 생성
Copy $\rightarrow$ 주소 공간만 복사하므로 부모 프로세스의 비동기 작업 등을 상속 x

*exec() : 새롭게 프로세스 생성하는 함수

$\rightarrow$ 새로운 프로세스

PID가 새로 프로세스 할당 $\rightarrow$ 동일하지 않음

생성 : 메모리 할당 (코드) + PCB 할당 + VMS 할당 (Heap, Stack, Data)

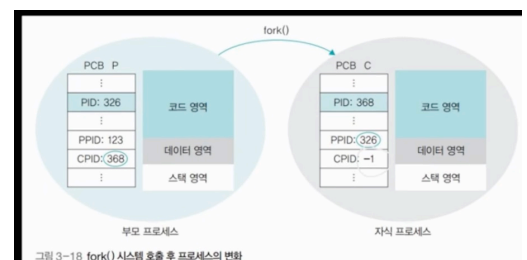


그림 3-18 fork() 시스템 호출 후 프로세스의 변화

✓ 대기 상태 (Ready)

- = 메모리 공간이 충분하면 메모리를 할당, 충분하지 않으면 아닌 상태로 대기
- = CPU 스케줄러로부터 CPU 소유권이 넘어오기를 기다리는 상태

✓ 대기 중단 상태 (Ready Suspended)

- = 메모리 부족으로 일시 중단된 상태

✓ 실행 상태 (Running)

- = CPU 소유권과 메모리를 할당받고 인스트럭션을 수행 중인 상태를 의미
- = CPU burst 발생

✓ 중단 상태(Blocked)

- = 어떤 이벤트 발생 후 기다리며 프로세스가 차단된 상태
- = I/O 디바이스에 의한 인터럽트로 많이 발생
- = ex) 프린트 인쇄 버튼 눌렀을때 프로세스 잠깐 로딩(멈춤) 현상 발생하는 경우

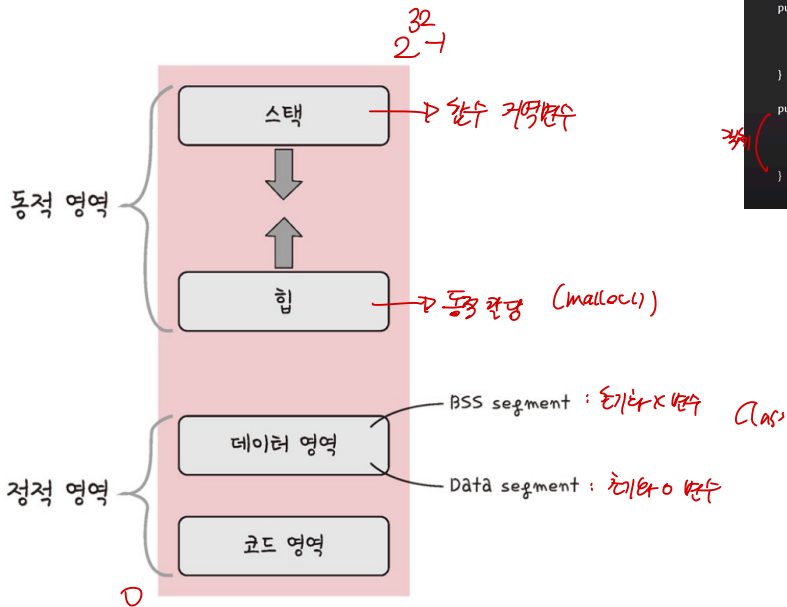
✓ 일시 중단 상태(Blocked Suspended)

- = 대기 중단과 유사
- = 프로세스가 실행되려고 했지만 메모리 부족으로 일시 중단된 상태

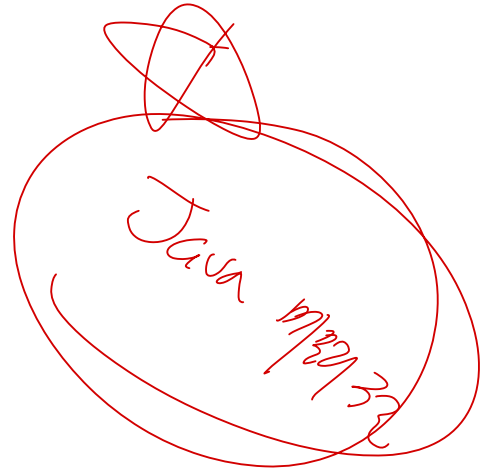
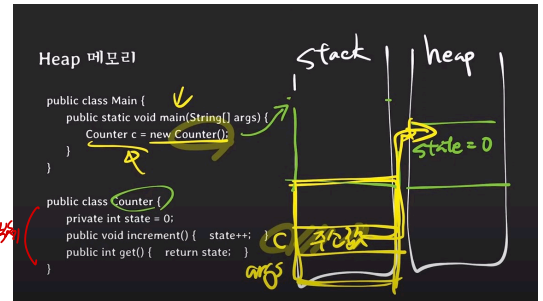
✓ 종료 상태 (Terminated)

- = 메모리와 CPU 소유권을 모두 놓고 가는 상태
- = 부모 프로세스 종료 / 자식 프로세스에 할당된 자원의 한계치를 넘을 경우 / 사용자가 process.kill 등 명령어로 프로세스 종료

3.3.3 프로세스의 메모리 구조



▲ 그림 3-19 프로세스의 메모리 구조



✓ 스택과 힙

- 동적 할당 (런타임 단계에서 메모리 할당)
- 스택 : 지역변수, 매개변수, 실행되는 함수에 의해 늘어되거나 줄어드는 메모리 영역
 - 함수가 호출될 때마다 호출될 때의 환경 등 특정 정보가 스택에 계속해서 저장
 - 재귀 함수가 호출될 경우
 - 새로운 스택 프레임이 매번 사용되기 때문에 함수 내의 변수 집합이 해당 함수의 다른 인스턴스 변수를 방해하지 않음
- 힙 : 동적으로 할당되는 변수들을 담음
 - malloc(), free() 를 통해 관리 가능
 - 동적으로 관리되는 자료 구조의 경우 힙 영역 사용
 - ex) vector는 내부적으로 힙 영역 사용

✓ 데이터 영역과 코드 영역

= 정적 할당되는 영역

= 데이터 영역 (BSS segment + Data Segment + code/text Segment)

- BSS segment 메모리 영역에 할당되는 변수들
 - 전역 변수 or static, const로 선언
 - 0으로 초기화 or 어떤 값으로도 초기화 되어 있지 않은 변수들

- Data segment 메모리 영역에 할당되는 변수들
 - 전역 변수 or static, const로 선언
 - 0이 아닌 값으로 초기화된 변수
- code segment : 프로그램 코드

PCB는 프로세스 스케줄링 상태, 프로세스 ID 등의 다음과 같은 정보로 이루어져 있습니다.

- 프로세스 스케줄링 상태: '준비', '일시중단' 등 프로세스가 CPU에 대한 소유권을 얻은 이후의 상태
- 프로세스 ID: 프로세스 ID, 해당 프로세스의 자식 프로세스 ID
- 프로세스 권한: 컴퓨터 자원 또는 I/O 디바이스에 대한 권한 정보
- 프로그램 카운터: 프로세스에서 실행해야 할 다음 명령어의 주소에 대한 포인터
- CPU 레지스터: 프로세스를 실행하기 위해 저장해야 할 레지스터에 대한 정보
- CPU 스케줄링 정보: CPU 스케줄러에 의해 중단된 시간 등에 대한 정보
- 계정 정보: 프로세스 실행에 사용된 CPU 사용량, 실행한 유저의 정보
- I/O 상태 정보: 프로세스에 할당된 I/O 디바이스 목록

3.3.3 PCB

✓ PCB란

= process control block

= OS 내부의 자료 구조

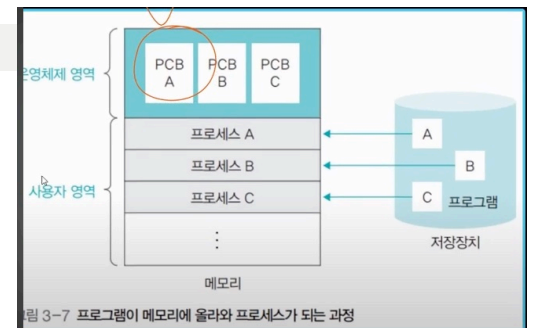
= 운영체제에서 프로세스에 대한 메타데이터를 저장한 데이터

= 프로세스 제어 블록

= 프로세스 생성시 OS는 해당 PCB를 생성

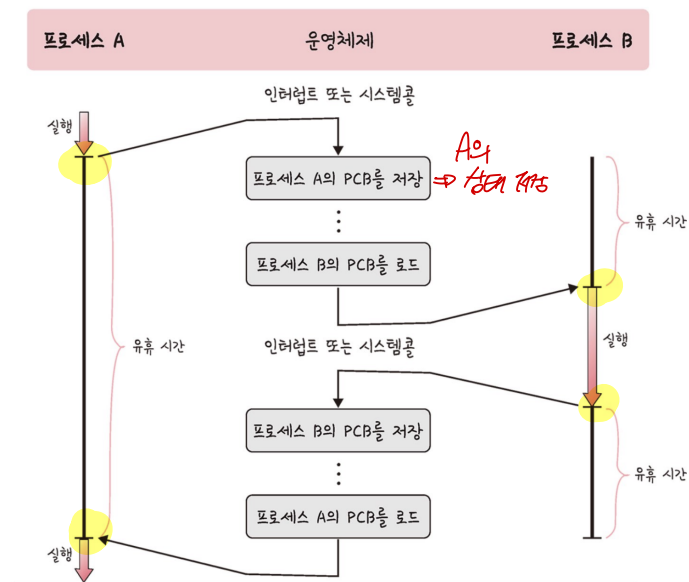
= 프로그램 실행시 프로세스가 생성되고 프로세스 주소 값들에 앞서 설명한 스택, 힙 등의 구조를 기반으로 메모리 할당

=

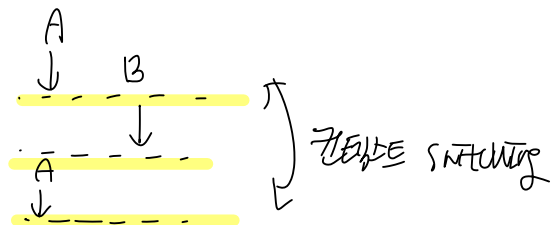


✓ 컨텍스트 스위칭

= PCB를 기반으로 프로세스의 상태를 저장하고 로드시키는 과정



▲ 그림 3-20 컨텍스트 스위칭



비용: 캐시미스

컨텍스트 스위칭이 일어날 때 프로세스가 가지고 있는 메모리 주소가 그대로 있으면 잘못된 주소변환이 생기므로 캐시클리어 과정을 겪게되고이 때문에 캐시미스가 발생합니다.

스레드에서의 컨텍스트 스위칭

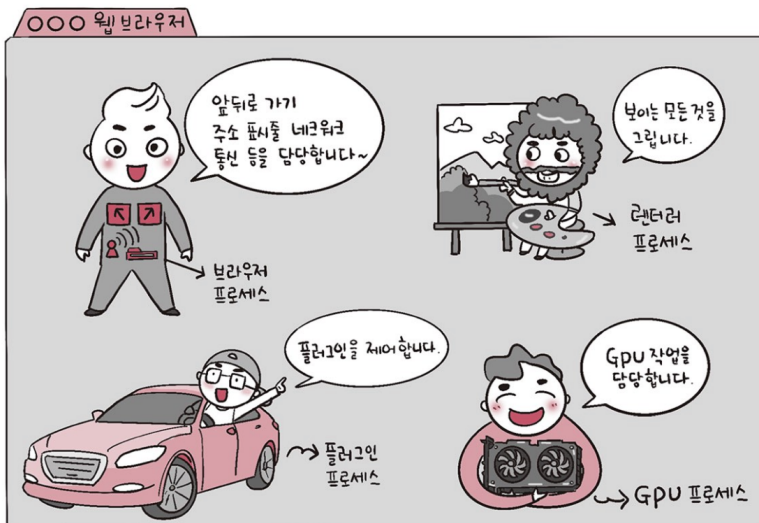
참고로이컨텍스트스위칭은스레드에서도일어납니다. 스레드는스택영역을제외한모든 메모리를공유하기때문에스레드컨텍스트스위칭의경우비용이더 적고시간도더적게걸립니다.

3.3.5 멀티프로세싱

멀티프로세싱은 여러 개의 '프로세스' → 멀티프로세스를 통해 동시에 2개 이상의 일을 수행하는 것

✓ 웹 브라우저

웹 브라우저는 멀티프로세스 구조를 가짐



▲ 그림 3-21 웹 브라우저의 멀티프로세스 구조

▪브라우저프로세스

- 주소표시줄, 북마크막대, 뒤로가기버튼, 앞으로가기 버튼 등 담당
- 네트워크 요청이나 파일 접근과 같은 권한 담당

▪렌더러 프로세스

- 웹사이트가 '보이는' 부분의 모든 것을 제어

▪플러그인 프로세스

- 웹사이트에서 사용하는 플러그인 제어

▪GPU프로세스

- GPU를 이용해 화면을 그리는 부분 제어



= 멀티 프로세스는 IPC(Inter Process Communication)가 가능

*IPC : 프로세스끼리 데이터를 주고 받고 공유 데이터를 관리하는 메커니즘

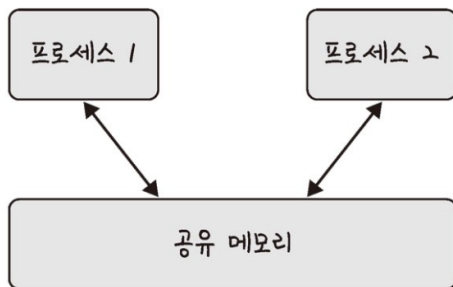
= ex) 클라이언트는 데이터를 요청하고 서버는 클라이언트 요청에 응답하는 상황

= 종류) 공유 메모리, 파일, 소켓, 익명 파이프, 명명 파이프, 메시지 큐

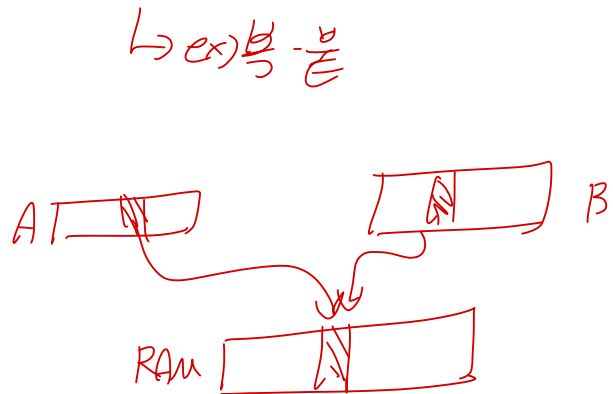
→ 메모리가 완전히 공유되는 스레드보다 속도는 떨어짐

▪공유 메모리

여러 프로세스에 동일한 메모리 블록에 대한 접근 권한이 부여되어 프로세스가 서로 통신할 수 있도록 공유 메모리를 생성해 통신하는 것



▲ 그림 3-22 공유 메모리



: 공유 메모리를 통해 여러 프로세스가 하나의 메모리를 공유할 수 있음

: 메모리 자체를 공유하므로 불필요한 데이터 복사의 오버헤드 발생x

→ 속도 빠름

→ 같은 메모리 영역을 여러 프로세스가 공유 → 동기화 필요

ex) 하드웨어 관점에서 공유 메모리는 CPU가 접근할 수 있는 큰 랜덤 접근 메모리 RAM을 가리키기도 함

■파일

파일은 디스크에 저장된 데이터 또는 파일 서버에서 제공한 데이터를 말함

→ 파일을 통해 프로세스 간 통신

ex) A 프로세스가 데이터를 파일에 쓰면, B 프로세스가 그 파일을 읽어서 데이터를 가져올 수 있습니다.

→ 직접 통신하지 않고 간접적으로 데이터를 공유. 데이터 교환이나 상태 공유를 할 수 있는 방식이 됩니다.

→ 파일
상징적 파일
→
서버 인스턴스

■소켓

동일한 컴퓨터의 다른 프로세스나 네트워크의 다른 컴퓨터로 네트워크 인터페이스를 통해 전송하는 데이터를 의미

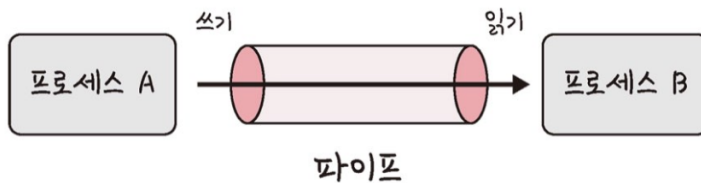
ex) TCP/UDP

■익명 파이프

프로세스 간에 FIFO 방식으로 읽히는 임시 공간인 파이프를 기반으로 데이터 주고 받음.

단방향 방식의 읽기 전용, 쓰기 전용 파이프를 만들어서 작동하는 방식

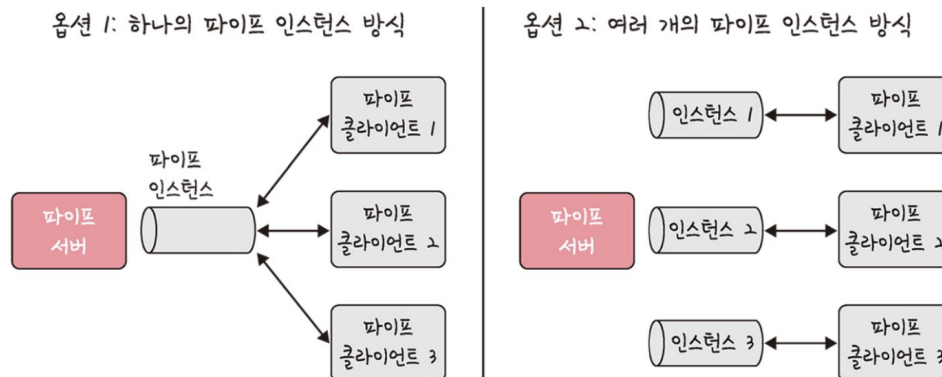
→ 부모, 자식 프로세스 간에만 사용 가능하고 다른 네트워크상에서는 사용 불가



▲ 그림 3-23 익명 파이프

■명명된 파이프

파이프 서버와 하나 이상의 파이프 클라이언트 간의 통신을 위한 명명된 단방향 또는 양방향 파이프



▲ 그림 3-24 명명된 파이프

- 클라이언트/서버 통신을 위한 별도의 파이프 제공
- 여러 파이프를 동시에 사용 가능
- 컴퓨터의 프로세스끼리 또는 다른 네트워크상의 컴퓨터와도 통신 가능함
- 서버용 파이프와 클라이언트용 파이프로 구분해서 작동, 1개 또는 n개의 인스턴스 기반으로 통신

■ 메시지 큐

메시지를 큐 데이터 구조 형태로 관리하는 것.

7



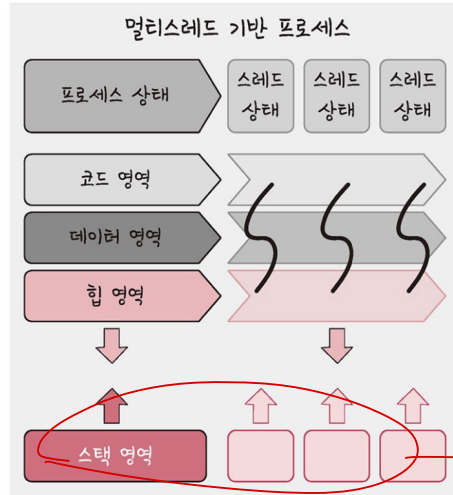
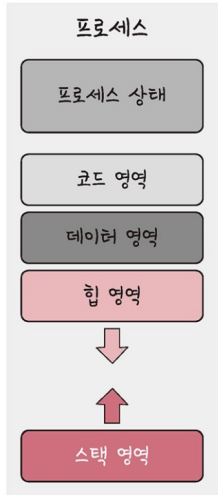
▲ 그림 3-25 메시지 큐

- 커널의 전역변수 형태 등 커널에서 전역적으로 관리
- 다른 IPC 방식에 비해 사용 방법이 직관적이고 간단
- 다른 코드의 수정 없이 단지 몇 줄의 코드를 추가시켜 간단하게 메시지 큐에 접근 가능
- 공유 메모리를 통해 IPC 구현시 쓰기 및 읽기 빈도 높을 경우 동기화로 인해 기능 구현 복잡해짐
 - 메시지 큐를 사용해 구현

3.3.6 스레드와 멀티스레딩

✅ 스레드

- = 프로세스의 실행 가능한 가장 작은 단위
- = 프로세스는 여러 스레드를 가질 수 있음



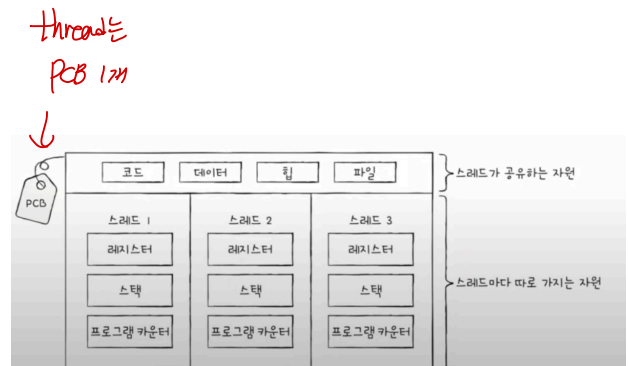
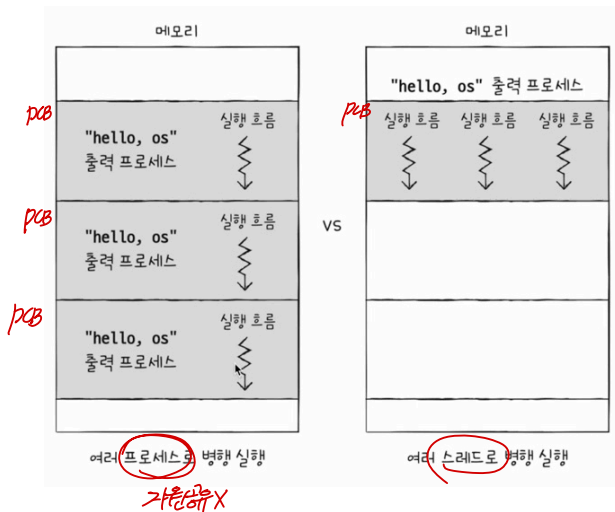
▲ 그림 3-26 프로세스와 멀티스레드

프로세스	스레드
코드, 데이터, 스택, 힙을 각각 생성	코드, 데이터, 힙을 서로 공유 그 외의 영역은 각각 생성

✓ 멀티스레딩

- = 프로세스 내 작업을 여러개의 스레드, 멀티스레드로 처리하는 기법
- = 스레드끼리 서로 자원을 공유하므로 효율성 높음

- 멀티스레드 종류 : 메인 스레드/ 워커 스레드/ 컴포지터 스레드/ 레스터 스레드





▲ 그림 3-27 웹 브라우저의 렌더러 프로세스를 이루는 스레드

3.3.7 공유 자원과 임계 영역

✓ 공유 자원

시스템 안에서 각 프로세스, 스레드가 함께 접근할 수 있는 모니터, 프린터, 메모리, 파일, 데이터 등의 자원이나 변수 등

- 경쟁 상태 : 2개 이상의 프로세스가 동시에 읽거나 쓰는 상황
 - 동시에 접근을 시도 할때 접근의 타이밍이나 순서 등이 결과값에 영향을 줄 수 있는 상태

✓ 임계 영역 (critical section)

둘 이상의 프로세스, 스레드가 공유 자원에 접근할 때 **순서 등의 이유로 결과가 달라지는 코드 영역**

→ (해결방안) 뮤텝스, 세마포어, 모니터 → 모두 상호 배제, 한정 대기, 융통성 조건 만족

→ 잠금 매커니즘 사용 (lock)

*상호배제 : 한 프로세스가 임계 영역에 들어갔을 때 다른 프로세스는 입장 불가

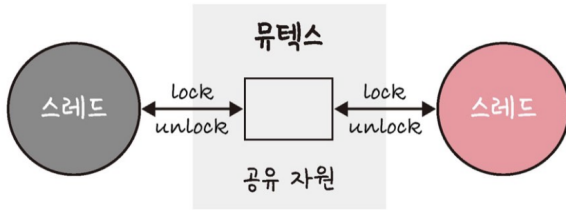
*한정대기 : 특정 프로세스가 영원히 임계 영역에 들어가지 못하면 안됨

*융통성 : 임계 영역 사용 없을 경우 임계 영역 외부의 어떠한 프로세스도 들어감 수 있음

→ 프로세스끼리 서로 방해하지 않음

• 뮤텝스

프로세스나 스레드가 공유 자원을 lock()을 통해 잠금 설정하고 사용한 후 unlock()을 통해 잠금 해제하는 객체



▲ 그림 3-29 뮤텝스

→ 잠금 설정시 다른 프로세스나 스레드는 잠긴 코드 영역에 접근할 수 없음

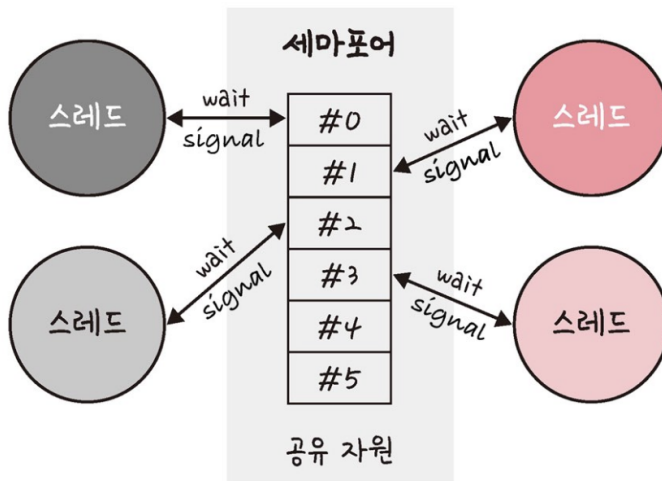
→ 뮤텝스는 잠금 or 잠금해제 상태만 가짐

■ 세마포어

일반화된 뮤텝스 → 간단한 정수 값과 두 가지 함수 wait 및 signal로 공유 자원에 대한 접근 처리

-세마포어에는 조건 변수 없음

-프로세스나 스레드가 세마포어 값 수정시 다른 프로세스나 스레드는 동시에 세마포어 값 수정 불가



▲ 그림 3-30 세마포어

- wait() 함수 (= p 함수) : 자신의 차례가 올 때 까지 기다리는 함수
- signal() 함수 (=v 함수) : 다음 프로세스로 순서를 넘겨주는 함수

프로세스나 스레드가 공유 자원 접근 → 세마포어에서 wait() 수행

프로세스나 스레드가 공유 자원 해제 → 세마포어에서 signal() 수행

바이너리 세마포어

0과 1 값만 가질 수 있는 세마포어

구현의 유사성으로 인해 뮤텍스는 바이너리 세마포어로 할 수 있지만 뮤텍스는 잠금 매커니즘이며 세마포어는 신호매커니즘

신호 매커니즘 = 노래 듣는 중 전화오면 노래 중지되고 통화 처리 작업 진행

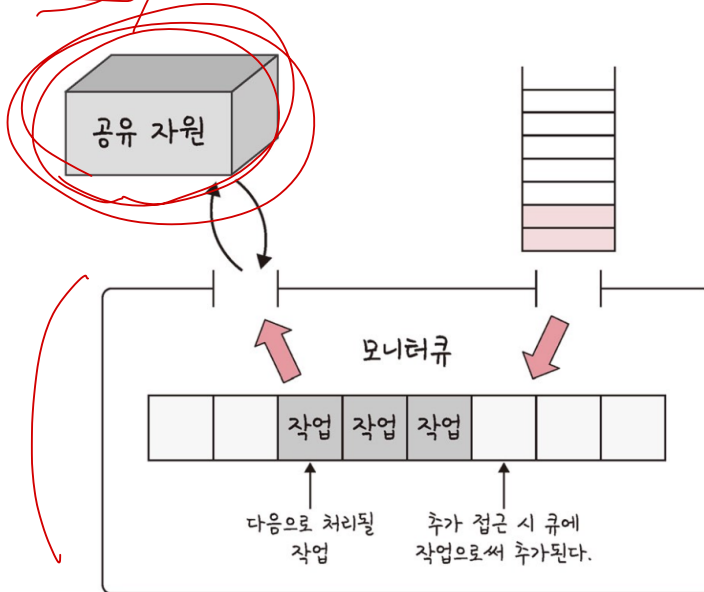
카운팅 세마포어

여러 개의 값을 가질 수 있는 세마포어

여러 자원에 대한 접근을 제어하는데 사용

모니터

둘 이상의 스레드나 프로세스가 공유 자원에 안전하게 접근할 수 있도록 공유 자원을 숨기고 해당 접근에 대해 인터페이스만 제공



▲ 그림 3-31 모니터

- 모니터큐를 통해 공유 자원에 대한 작업들을 순차적으로 처리
- 모니터는 세마포어보다 구현하기 쉬움
- 상호 배제는 자동이므로 구현 필요 없음

3.3.8 교착 상태

두 개 이상의 프로세스들이 서로가 가진 자원을 기다리며 중단된 상태

ex) 프로세스 A가 B에게 자원 요청시 B도 A가 점유하고 있는 자원 요청할 경우

✓ 교착 상태의 원인

1. 상호 배제 : 한 프로세스가 자원을 독점하고 있으며 다른 프로세스들은 접근 불가
2. 점유 대기 : 특정 프로세스가 점유한 자원을 다른 프로세스가 요청하는 상태
3. 비선점 : 다른 프로세스의 자원을 강제적으로 가져올 수 없음
4. 환형 대기 : 프로세스 A는 프로세스 B의 자원을 요구하고, 프로세스 B는 프로세스 A의 자원을 요구하는 등 서로가 서로의 자원을 요구하는 상황

✓ 교착 상태 해결 방법

1. 자원을 할당할 때 애초에 조건이 성립되지 않도록 설계
2. 교착 상태 가능성이 없을 때만 자원 할당되며, 프로세스당 요청할 자원들의 최대치를 통해 자원 할당 가능 여부를 파악하는 '은행원 알고리즘' 사용
3. 교착 상태가 발생하면 사이클이 있는지 찾아보고 이에 관련된 프로세스를 한개 씩 지움
4. 교착 상태 처리 비용은 비싸고 드물기 때문에 사용자의 작업을 종료 시킴. (현대 os 방식)